

Diagramă microservicii

1. Identificăm tier-urile

ex: Client → Web → Logic → Persistence → Data

2. Plăsam fiecare componentă în tier-ul potrivit:

ex: → vorbește cu utilizatorul? → Web Tier

→ procesează logică? → Logic Tier

→ gestionează date? → Persistence Tier

3. Verificăm direcția dependențelor

ex: → Web apează Logic

→ Logic apează Persistence] → mică dată invers!

4. Etichetare săgeți

ex: Validează date
persistență

citire / scrie

Diagramă clase

1. Ce face aplicația?

ex: Aplicația primește date despre un student dintr-un formular web, le validează, le salvează într-un fișier XML și permite citirea și afișarea lor ulterioară.

Student Bean (pointing to student)

pag. web formular.jsp (pointing to formular web)

Student Validator (pointing to validează)

fișier pe disc student.xml (pointing to fișier XML)

repository (pointing to permite citirea și afișarea)

2. Extragem cuvinte pt a vedea clasele candidate.

ex: student → Student Bean

servlet → Process Student Servlet
Read Student Servlet
Hello Servlet

validator → Student Validator

repository → IStudentRepository (interfață)
Kml Student Repository (implementare)

3. Extragem verbele pt a vedea metodele și relațiile.

ex: preia date → doGet (req, res)

validarea → validate (student)

salvează → save (student)

citeste → load()

doGet (req, res)

afiseaza → forward (req, res)

4. Aplicăm principiile SOLID

ex: punem întrebarea: Dacă schimb ceva în sistem, în câte locuri trebuie să modific?

Ce s-ar putea schimba în viitor?

5. Stabilim relațiile între clase în funcție de atribute și metode.